CHAPTER 1

PRELIMINARIES

Why study programming languages?

- Working knowledge of one or two programming languages is not sufficient for a computer scientists
- You should study general concepts of language design and evaluation

Reasons for Studying Principles of Programming Languages

- Increased capacity to express ideas
- Improved background for choosing appropriate languages
- Increased ability to learn new languages
- Better understanding of significance of implementation
- Better use of languages that are already known
- Overall advancement of computing

Increased capacity to express ideas

- Consider natural languages. The depth at which you can think is influenced by the expressive power of the language that you are using to communicate.
- It is similar for programming languages. The language in which you develop software puts limits on the kind of control structures, data structures and abstractions, and therefore the algorithms that you can develop.

Increased capacity to express ideas

- Awareness of a wider variety of programming language features can reduce such limitations in software development.
- If you have to use a specific language which does not have those capabilities, still you can make use of those concepts by simulating them.
- For example, you can simulate associative arrays (maps, dictionaries) with the structures in C

Improved background for choosing appropriate languages

- If all you have is a hammer, then everything looks like a nail.
- Same for PLs. If all you know is Java, then every solution to every problem will be a Java solution.
- Many programmers learn one or two languages specific to the projects. Some of those languages may be no longer used.
- When they start a new project, they continue to use those languages which are old and not suited to the current projects. However, another language may be more appropriate.

Improved background for choosing appropriate languages

- If they were familiar with the other languages, particularly the features in those languages, they would choose better languages
- Studying the principles of PLs provides a way to judge languages, and make informed statements:
  - "The advantages of Python for this problem are …..",
  - "The advantages of Java are …."

Increased ability to learn new languages

- If you know the PL concepts, you will learn other languages much easier compared to programmers knowing only one or two languages.
- For example, if you know concept of object-oriented programming, it is easier to learn C++ after learning Java
- Just like natural languages, learning new languages actually causes you to learn things about the languages you already know
- Example: Languages may differ in the way in which arithmetic expressions are evaluated

Better understanding of significance of implementation

- The best programmers are the ones having at least understanding of how things work in the background
- You can simply write a code and let the compiler do everything, but knowing implementation details helps you to use a language more intelligently and write the code that is more efficient
- Also, it allows you to visualize how a computer executes language constructs (e.g. recursion is slow)

Programming Domains

- Scientific applications(first digital computers – 1940s)
  - Large floating-point arithmetic, execution efficiency, arrays and matrices, counting loops
  - Iterations are more important than recursion
  - **Examples: FORTRAN, C, R, Python, Julia, Matlab, …**
- Business applications (1950s)
  - Produce reports, use decimal numbers and characters
  - **Examples: COBOL, SQL, Python, Java, C lang, Cobalt, JavaScript, …**
- Artificial intelligence
  - Symbols rather than numbers manipulated; use of linked lists rather than arrays
  - **Examples: Python, Lisp, Java, Prolog, Julia, Scala, …**
- Systems programming
  - Operating system and all the programming support tools of a computer system
  - Efficient and fast execution, low-level features for peripheral device drivers
  - **Examples: PLS/2, BLISS, C**
- Web Software
  - A collection of languages: **JavaScript, PHP, Java, Python, Swift, Kotlin, Ruby, …**

Top Programming Languages Of 2023

The best programming languages to learn in 2022

History of Programming Languages

Why are there so many programming languages?

- evolution
  - we've learned better ways of doing things over time
- socio-economic factors
  - proprietary interests, commercial advantage
- orientation towards special purposes
- orientation towards special hardware
- diverse ideas about what is pleasant to use

Language Evaluation Criteria

- **Readability**: the ease with which programs can be read and understood
- **Writability**: the ease with which a language can be used to create programs
- **Reliability**: conformance to specifications (i.e., performs to its specifications)
- **Cost**: the ultimate total cost

Evaluation Criteria: Readability

Readability is important for maintenance

- Characteristics that contribute to Readability:
  - Overall simplicity
  - Orthogonality
  - Control statements
  - Data types and structures
  - Syntax Considerations

Overall Simplicity

- **A manageable set of features and constructs**
  - User learns only a subset
    - but this subset may differ from one user to another
- ***Minimal Feature Multiplicity:*** having more than one way to accomplish an operation
  - ***e.g.*** In Java

```
count = count + 1
count += 1
count ++
++count
```

- ***Minimal Operator Overloading***
  - if users are allowed to create their own operator and not use this sensibly, it is a problem
  - ***e.g.*** to use **+** for **integer and floating point addition** is acceptable, but **to sum up all the elements of two singe dimensional arrays** is not – different from vector addition

On the other hand, the simplest does not mean the best.
e.g. Assembly languages

Orthogonality

- A relatively small set of primitive constructs can be combined in a relatively small number of ways to build the control and data structures of the language

- Every possible combination of primitives is legal and meaningful

- **Example:**
  - *Four primitive data types:* integer, float, double and character
  - *Two type operators:* array and pointer

  - If the two type operators can be applied to themselves and the four primitive data types, a large number of data structures can be defined
  - However, if pointers were not allowed to point to arrays, many of those possibilities would be eliminated

Orthogonality

- **Example:** *Adding two 32-bit integers residing in memory or registers, and replacing one of them with the sum:*
  - IBM (Mainframe) Assembly language has two instructions:
    - **A** Register1, MemoryCell1
      - Register1 ← contents(Register1) + contents(MemoryCell1)
    - **AR** Register1, Register2
      - Register1 ← contents(Register1) + contents(Register2)
  - Intel x86-64 Assembly language has one instruction:
    - **addq** operand1, operand2
      - Operand2 ← contents(operand1) + contents(operand2)
      - Here, either operand can be a register or a memory cell

Orthogonality

- Intel instruction design is orthogonal
  - Because a single instruction can use either registers or memory cells as the operands
- IBM design is not orthogonal
  - Only two operand combinations are legal out of four possibilities and require different instructions, A and AR
  - More restricted and less writable (you cannot add two values and store the sum in memory location)
  - More difficult to learn because of the restrictions and the additional instructions

Orthogonality

- Orthogonality is closely related to simplicity
- The more orthogonal the design of a language, the fewer exceptions the language rules require

- **C is not an orthogonal language**, because
  - records(structs) can be returned from functions, but arrays cannot
  - a member of a structure can be any type but not void or structure of the same type
  - a member of an array can be any type but not void or function

Orthogonality

- Too much orthogonality can cause problems as well:

- **ALGOL68 is the most orthogonal language.**
  - Every construct has a type
  - Most constructs produce values
  - This may result in extremely complex constructs,
  - e.g., A conditional can appear as the left side of an assignment statement, as long as it produces a location:

```
(if (A<B) then C else D) := 3
```

  - This extreme form of orthogonality leads to unnecessary complexity

- **Functional languages** offer a good combination of simplicity and orthogonality.

Control Statements

- Control statement design of a language is an important factor in readability.

- For example, languages such as **BASIC** and **FORTRAN**, that allow the use of **goto** statement, are **not readable**.
- If C did not have a loop construct:

```
loop1:
    if (incr >=20) go to out;
loop2:
    if (sum > 100) go to next;
    sum += incr;
    go to loop2;
next:
    incr++;
    go to loop1:
out:
```

```
while (incr < 20){
    while (sum < =100){
        sum += incr;
    }
    incr++;
}
```

Data types and structures

- Facilities for defining data types and data structures are helpful for readability
  - If there is no `boolean` type available then a flag may be defined as integer:
    - `found = 1` (instead of `found = true`)
  - May mean something is found as boolean or what is found is 1
- An array of record type is more readable than a set of independent arrays
  - In Fortran

```
Character(Len=30) Name(100)
IntegerEmployeeNumber(100)
Real Salary(100)
```

Syntax considerations

- **Identifier Forms:** restricting identifier length is bad for readability.
- Example:
  - FORTRAN77 identifiers can have at most 6 characters.
  - The extreme case is the ANSI BASIC, where an identifier is either a single character or a single character followed by a single digit.
- Availability of word concatenating characters (e.g., _) is good for readability.

Syntax considerations

- **Special Words:** Readability is increased by special words (e.g., **`begin`**, **`end`**, **`for`**).

- In PASCAL and C, `end` or `}` is used to end a compound statement. It is difficult to tell what an **`end`** or **`}`** terminates.

- However, ADA uses **`end if`** and **`end loop`** to terminate a selection and a loop, respectively.

- Another issue is the use of special words as names of variables.
  - For example, in FORTRAN77, special words, e.g., **DO** and **END** can be used as variable names.

Evaluation Criteria: Writability

- Ease of creating programs
- Characteristics that contribute to readability
  - Simplicity and orthogonality
    - Few constructs, a small number of primitives, a small set of rules for combining them
  - Support for abstraction
    - The ability to define and use complex structures or operations in ways that allow details to be ignored
  - Expressivity
    - A set of relatively convenient ways of specifying operations
    - Strength and number of operators and predefined functions

Support for abstraction

- **Abstraction:** ability to define and use complicated structures and operations in ways that allows ignoring the details.
- Abstraction is the key concept in contemporary programming languages
- The degree of abstraction allowed by a programming language and the naturalness of its expressions are very important to its writability.
- PLs can support two types of abstraction:
  - process
  - data

Process Abstraction

- The simplest example of abstraction is subprograms (e.g., methods).
- You define a subprogram, then use it by ignoring how it actually works.
- Eliminates replication of the code
- Ignores the implementation details
- e.g. sort algorithm

Data Abstraction

- As an example of data abstraction, a tree can be represented more naturally using pointers in nodes.
- In FORTRAN77, where pointer types are not available, a tree can be represented using 3 parallel arrays, two of which contain the indexes of the offspring, and the last one containing the data.

Expressivity

- Having more convenient and shorter ways of specifying computations.
- For example,
  - `count++;` is more convenient and expressive than

    ```
    count = count + 1;
    ```

  - **`for`** is more easier to write loops than **`while`**

Evaluation Criteria: Reliability

- **Reliable**: it performs to its specifications under all conditions
- Characteristics that contribute to Reliability:
  - Type checking
    - Testing for type errors
  - Exception handling
    - Intercept run-time errors and take corrective measures
  - Aliasing
    - Presence of two or more distinct referencing methods for the same memory location
  - Readability and writability
    - A language that does not support “natural” ways of expressing an algorithm will require the use of “unnatural” approaches, and hence reduced reliability

Type Checking

- Testing for type errors in a given program either by the compiler or during program execution
- The compatibility between two variables or a variable and a constant that are somehow related (e.g., assignment, argument of an operation, formal and actual parameters of a method).
- **Run-time (Execution-time)** checking is expensive.
- **Compile-time** checking is more desirable.
- The earlier the errors in programs are detected, the less expensive it is to make the required repairs

Type Checking

- Ada requires type checking on all variables in compile time.
- Original C language required no type checking neither in compilation nor execution time. That can cause many problems.
- Java requires checks of the types of nearly all variables and expressions at compile time

Type Checking

- For example, the following program compiles and runs!

```
foo (float a) {
   printf ("a: %g and square(a): %g\n", a, a*a);
}
main () {
   char z = 'b';
   foo(z);
}
```

Output is : a: 98 and square(a): 9604

Type Checking

- **Subscript range** checking for arrays is a part of type checking, but it must be done in the **run-time**.
- Out-of-range subscript often cause errors that do not appear until long after actual violation.

Exception Handling

- The ability of a program
  - to intercept run-time errors, as well as other unusual conditions
  - to take corrective measures and continue
- Ada, C++, and Java include extensive capabilities for exception handling, but in C and Fortran it is practically non-existent

Aliasing

- Having two distinct referencing methods (or names) for the same memory cell.
- It is a dangerous feature in a programming language.
- e.g., pointers in C
  - two different variables can refer to the same memory cell

Readability and Writability

- The easier a program to write, the more likely it is correct.
- Programs that are difficult to read are difficult both to write and modify.

Evaluation Criteria: Cost

- Training programmers to use the language
  - Function of simplicity and orthogonality,
  - Experience of the programmers.
- Writing programs (closeness to particular applications)
  - Function of the writability
- Compiling programs
  - cost of compiler, and time to compile

Evaluation Criteria: Cost

- Executing programs
  - If a language requires many runtime type checking, the programs written in that language will execute slowly.
  - **Trade-off** between compilation cost and execution cost.
  - **Optimization:** decreases the size or increases the execution speed.
  - Without optimization, compilation cost can be reduced.
  - Extra compilation effort can result in faster execution.
- Language implementation system: availability of free compilers
  - If expensive or runs only on expensive hardware it will not be widely used

Evaluation Criteria: Cost

- Reliability: poor reliability leads to high costs
  - important for critical systems such as a power plant or X-ray machine
- Maintaining programs
  - For corrections, modifications and additions.
  - Function of readability.
  - Usually, and unfortunately, maintenance is done by people other than the original authors of the program
  - For large programs, the maintenance cost is about 2 to 4 times the development costs.

Evaluation Criteria: Others

- Portability
  - The ease with which programs can be moved from one implementation to another
- Generality
  - The applicability to a wide range of applications
- Well-definedness
  - The completeness and precision of the language's official definition

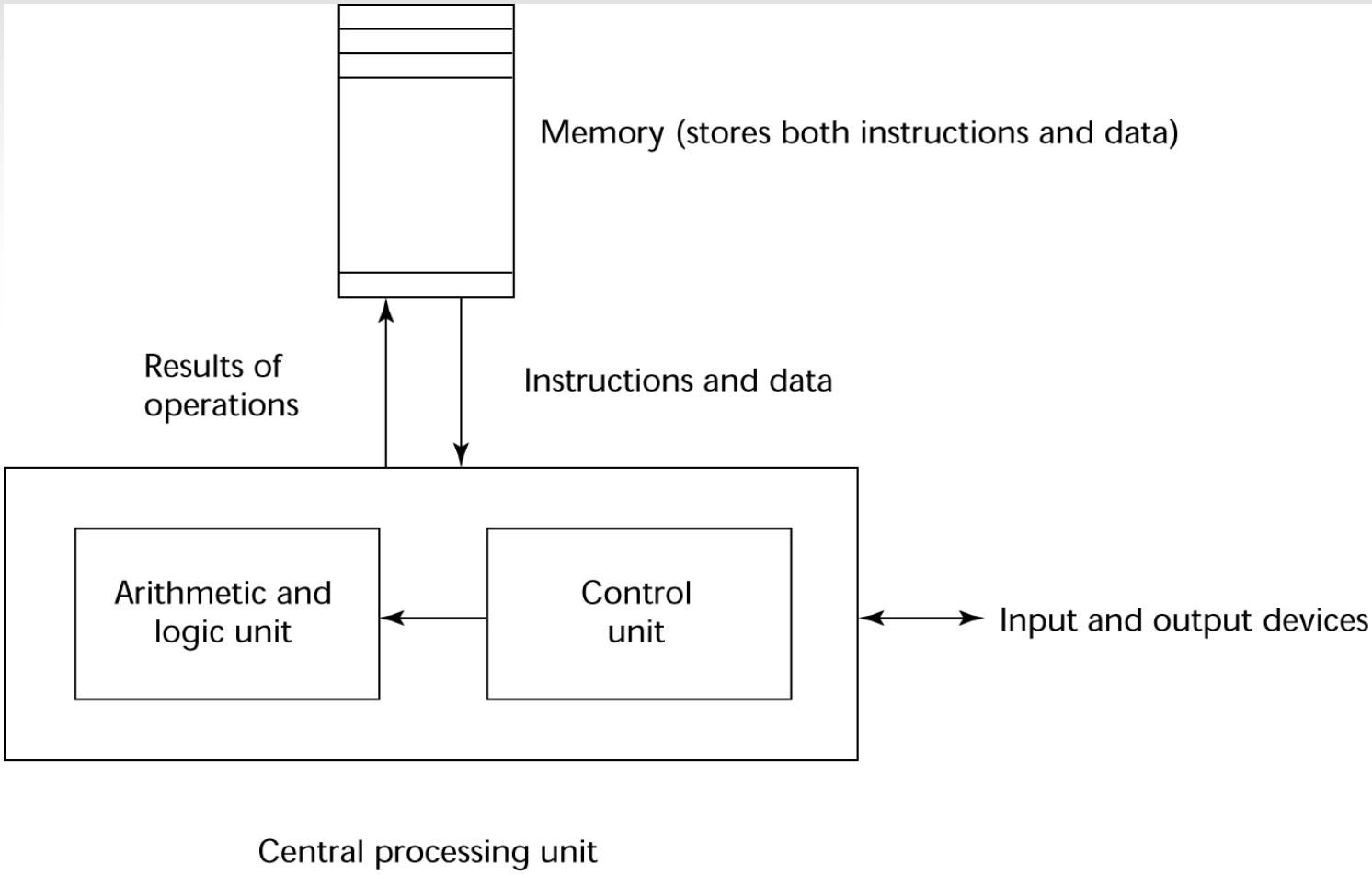
Influences on Language Design

- Several other factors influence the design of PLs
  - Computer Architecture
    - Languages are developed around the prevalent computer architecture, known as the *von Neumann* architecture
  - Programming Methodologies
    - New software development methodologies (e.g., object-oriented software development) led to new programming paradigms and by extension, new programming languages

Computer Architecture Influence

- Well-known computer architecture: Von Neumann
- Imperative languages, most dominant, because of von Neumann computers
  - Data and programs stored in memory
  - Memory is separate from CPU
  - Instructions and data are piped from memory to CPU
  - Basis for imperative languages
    - Variables model memory cells
    - Assignment statements model piping
    - Iteration is efficient

The von Neumann Architecture



The von Neumann Architecture

Fetch-execute-cycle (on a von Neumann architecture computer)

```
initialize the program counter
repeat forever
  fetch the instruction pointed by the counter
  increment the counter
  decode the instruction
  execute the instruction
end repeat
```

Programming Methodologies

- There has been a major shift from process-oriented programming to data-oriented programming.
- More emphasis on Abstract Data Types.
- The latest trend in the evaluation of data-oriented software development is the object-oriented design.
  - It is based on Data Abstraction, Encapsulation, Information hiding, Dynamic type binding.
  - Smalltalk, Objective C, C++, Java are examples of object-oriented programming languages.

Language Categories

- Imperative
  - Command-driven or statement-oriented languages
  - Basic concept is the machine state (the set of all values for all memory locations)
  - A program consists of a sequence of statements and the execution of each statement changes the machine state
  - Programs take form:

```
statement1;
statement2;
… …
```

  - Examples: C, Fortran, Basic, Java, Perl, JavaScript, Visual BASIC .NET, C++

Language Categories

- Functional
  - Looks at the program as a set of functions that produce one result or another
  - The key question is: given the data and function, what will the program produce
  - Examples: LISP, Scheme, ML, Haskel
  - Statements take form:

$$\texttt{function}_n(\texttt{function}_1, \texttt{function}_2, \ldots (\texttt{data}))$$

Language Categories

- Logic / Rule-based
  - Execute checking to see if a particular condition is true and if so, perform the appropriate actions
  - The enabling conditions are usually written in terms of predicate calculus and take the form:

    $\texttt{condition}_1$ –> $\texttt{action}_1$

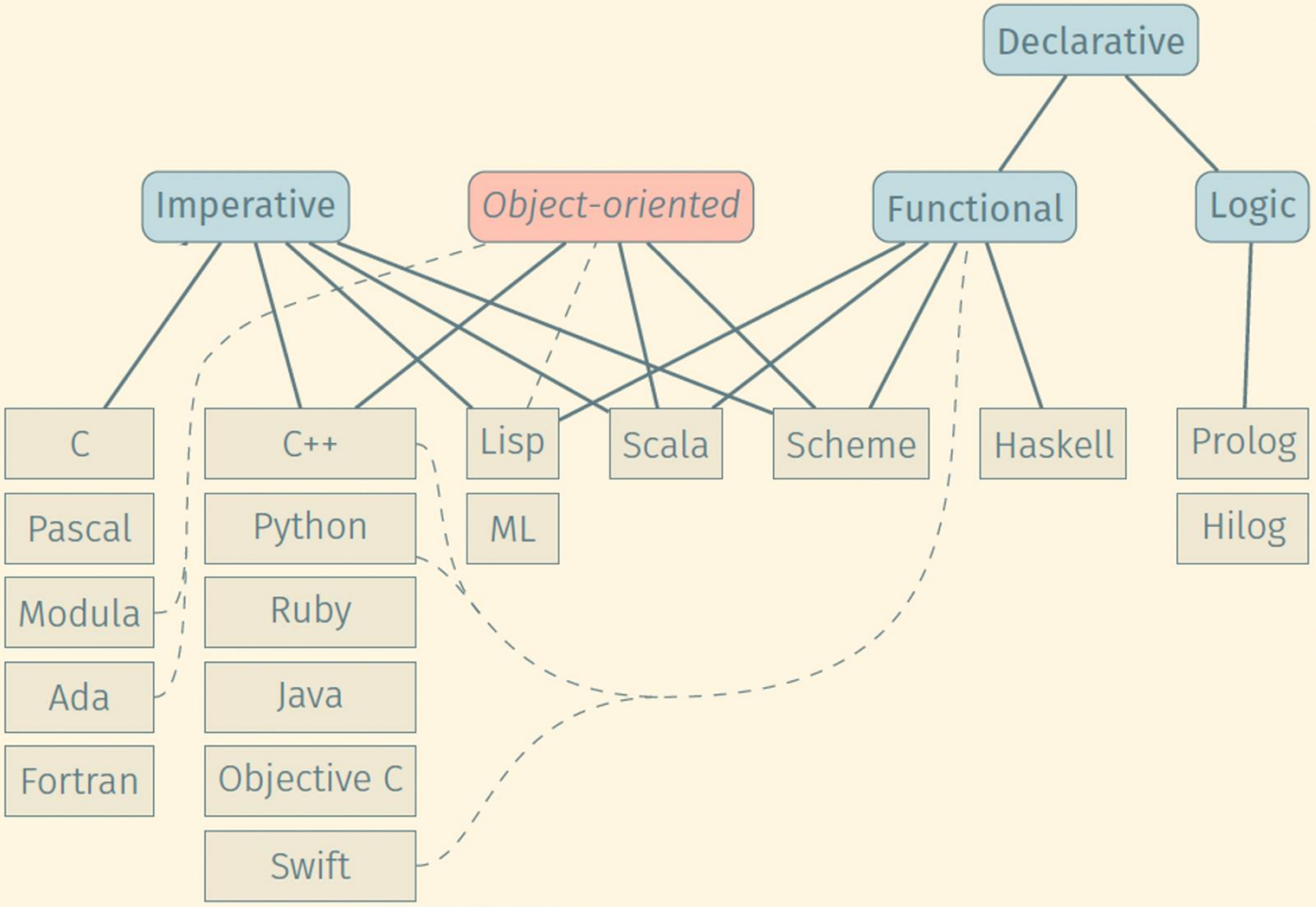
    $\texttt{condition}_2$ –> $\texttt{action}_2$

    … …
  - Examples: Prolog

Language Categories

- Object-oriented
  - Data structures and algorithms support data abstraction and allow programmer to use data in a fashion that closely represents its real-world use
  - Data abstraction is implemented by use of:
    - Encapsulation: data and procedures belonging to a class can only be accessed by that classes
    - Polymorphism: the same functions and operators can mean different thing depending on the parameters or operands
    - Inheritance: new classes may be defined in terms of other, simpler classes
  - Examples: Java, C++, …

TYPES OF PROGRAMMING LANGUAGES



IMPERATIVE VS DECLARATIVE PROGRAMMING

Imperative programming:

- Focus on telling the computer exactly which steps to execute
- Close to the machine
- Difficult to analyze/ automatically optimize
- Functions called for
  - Return values
  - Side effects

Declarative programming:

- Focus on telling the computer ***what*** to do, ***not how*** to do it
- More high-level, elegant, expressive
- Need to include evaluation engine in run-time system
- Can come with performance penalties
- Easier to analyze/automatically optimize/parallelize

FUNCTIONAL PROGRAMMING

- Functions have no side effects
- Variables are immutable once defined
- Functions are first-class objects
- Can express imperative computations elegantly!

Pros:

- Easier to analyze/optimize
- No specified execution order
  $\rightarrow$ easy to parallelize

Cons:

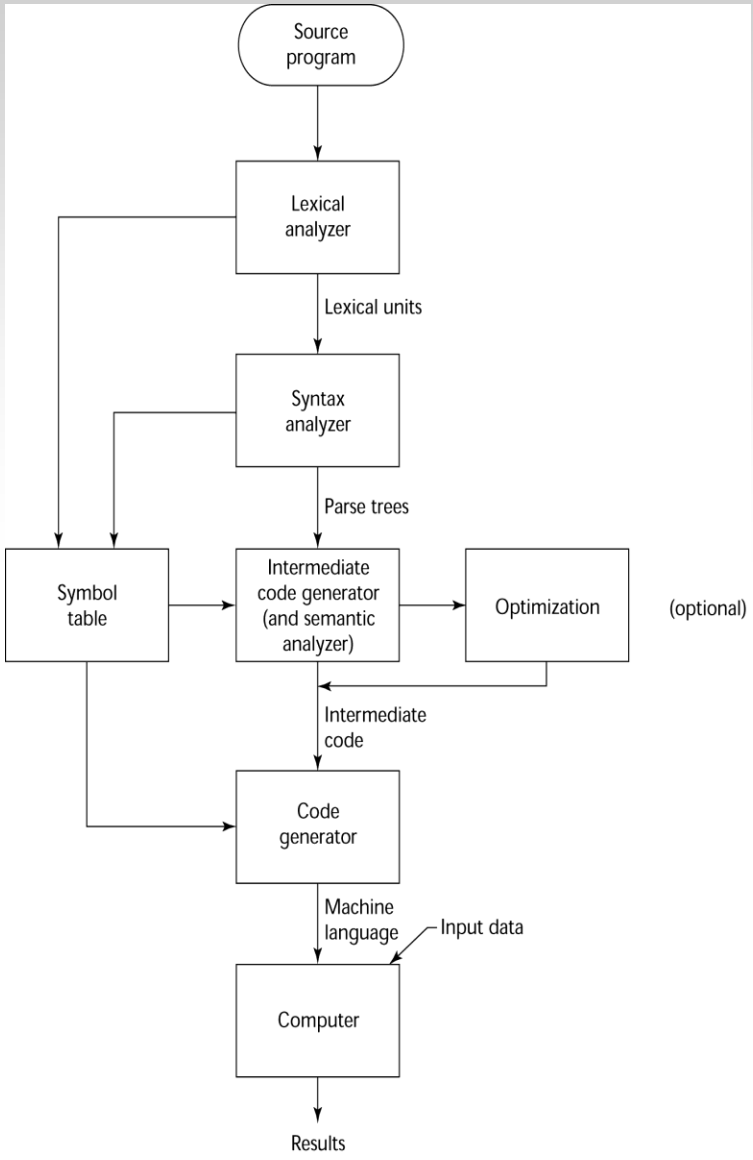
- Can be less efficient than well-designed imperative code
- Some imperative data structures are inherently more efficient than their purely functional counterparts

Implementation Methods

- Compilation
  - Programs are translated into machine language; includes JIT systems
  - Use: Large commercial applications
- Pure Interpretation
  - Programs are interpreted by another program known as an interpreter
  - Use: Small programs or when efficiency is not an issue
- Hybrid Implementation Systems
  - A compromise between compilers and pure interpreters
  - Use: Small and medium systems when efficiency is not the first concern

Compilation

- Translate high-level program (source language) into machine code (machine language)
- Fast execution
- Compilation process has several phases:
  - **Lexical Analysis:** converts characters in the source program into lexical units
  - **Syntax Analysis:** transforms lexical units into parse trees which represent the syntactic structure of program
  - **Intermediate Code Generator:** produces a program at an intermediate level. Similar to assembly languages
  - **Semantics Analysis:** checks for errors that are difficult to check during syntax analysis, such as type errors
  - **Optimization:** (optional) used if execution speed is more important than compilation speed
  - **Code generation:** machine code is generated
  - **Symbol table:** serves as a database of type and attribute information of each user defined name in a program. Placed by lexical and syntax analyzers, and used by semantic analyzer and code generator



Linker

- Most user programs require programs from the operating system, such as input/output
- Before the machine language programs produced by a compiler can be executed, the required programs from the operating system must be found and linked to user programs
- The linking operation connects the user program to the system programs by placing the addresses of the entry points of the system programs in the calls to them in the user programs
- User programs must often be linked to previously compiled user programs that reside in libraries

Fetch-Execute Cycle

- Execution of machine code on a von Neumann architecture computer occurs in a process called the **fetch-execute cycle**
- Programs reside in memory but are executed in the CPU
- Each instruction to be executed must be moved from memory to CPU
- The address of the next instruction to be executed is maintained in a register called the **program counter**

Fetch-Execute Cycle

```
Initialize the program counter
repeat forever
    fetch the instruction pointed to by the program counter
    increment the program counter to point at the next instruction
    decode the instruction
    execute the instruction
End repeat
```

Program terminates

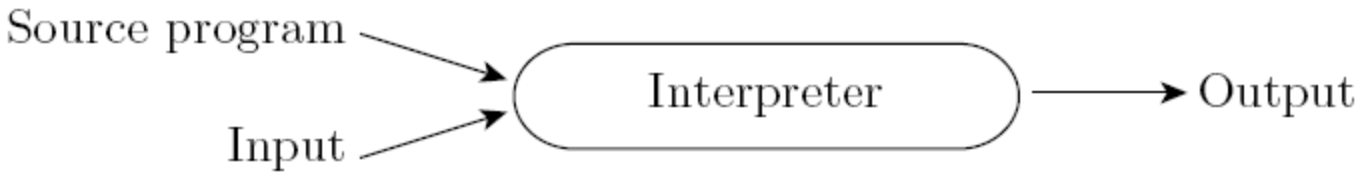
- when stop instruction is encountered

OR

- control transfers from user program to OS when the user program execution is complete

Pure Interpretation

- Each statement is decoded and executed individually .
- No translation
- The interpreter program acts as a software simulation of a machine whose fetch-execute cycle deals with high level language program statements rather than machine instructions
- Easier implementation of programs (run-time errors can easily and immediately be displayed)
- Slower execution (10 to 100 times slower than compiled programs)
- Often requires more space
- Some web scripting languages (e.g., JavaScript, PHP)



Interpretation vs. Compilation

How an English speaker can communicate to a French speaker

- The English speaker employs an ***interpreter*** who translates the English sentences into French as they are spoken. The English speaker says a sentence in English, the interpreter hears it, translates it in his/her brain into French, and the speaks the sentence in French. This is repeated for each sentence.
  - Progress is slow: there are pauses between sentences as the translation process takes place.
- The English speaker writes down (in English) what needs to be said. The whole document is then translated into French, producing another piece of paper with the French version written on it. There are two factors to consider in this scenario:
  - There is a slight delay at the start as the English document needs to be translated in its entity before it can be read.
  - The translated document can be read at any time after that, and at the normal speed at which the French-speaker reads.

Interpretation vs. Compilation

- An ***interpreter***, on the other hand, provides a means by which a program written in source language can be understood and executed by the CPU line by line. As the first line is encountered by the interpreter, it is translated and executed. Then it moves to the next line of source code and repeats the process
  - The interpreter is a program which is loaded into memory alongside the source program
  - Statements from the source program are fetched and executed one by one
  - No copy of the translation exists, and if the program is to be re-run, it has to be interpreted all over again.
- A ***compiler*** translates a complete source program into machine code. The whole source code file is compiled in one go, and a complete, compiled version of the file is produced. This can be saved on some secondary storage medium (e.g. floppy disk, hard disk...)
  - The program can only be executed once translation is complete
  - ANY changes to the source code require a complete recompilation.

Interpretation vs. Compilation

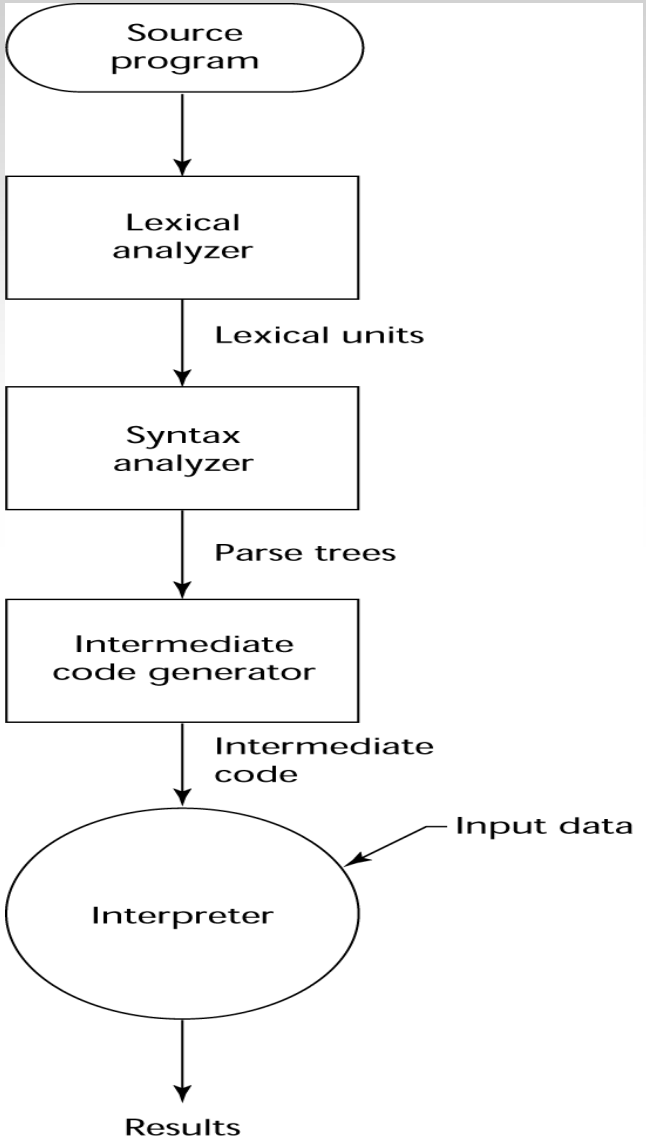
- Because the *interpreter* must be loaded into memory, there is less space available during execution; a *compiler* is only loaded into memory during compilation stage, and so only the machine code is resident in memory during run-time
- Once we have a *compiled* file, we can re-run it any time we want to without having to use the compiler each time; With any *interpreted* language, however, we would have to re-interpret the program each time we wanted to run it
- *Machine code* programs run more quickly than *interpreted* programs
- However, it is often quicker and easier to make changes with an *interpreted* program than a *compiled* one, and as such development time may be reduced.

Compilation vs. Interpretation

Compiler	Interpreter
• A compiler takes the entire program in one go.	• An interpreter takes a single line of code at a time.
• The compiler generates an intermediate machine code.	• The interpreter never produces any intermediate machine code.
• The compiler is best suited for the production environment.	• An interpreter is best suited for a software development environment.
• The compiler is used by programming languages such as C, C ++, C #, Scala, Java, etc.	• An interpreter is used by programming languages such as Python, PHP, Perl, Ruby, etc.

Hybrid Implementation Systems

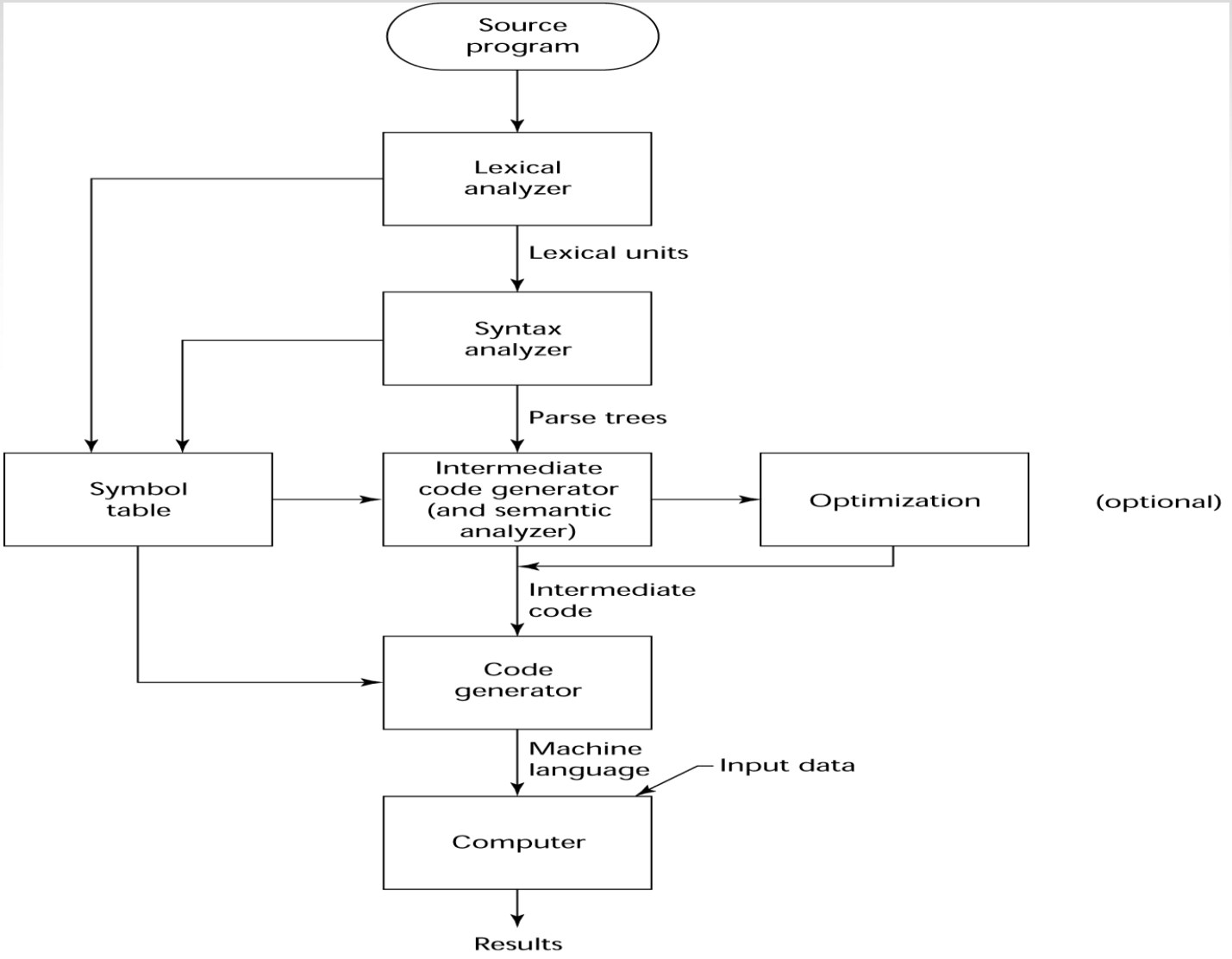
- A compromise between compilers and pure interpreters
- A high-level language program is translated to an intermediate language that allows easy interpretation
- Faster than pure interpretation
- Examples
  - Perl programs are partially compiled to detect errors before interpretation
  - Initial implementations of Java were hybrid; the intermediate form, *byte code*, provides portability to any machine that has a byte code interpreter and a run-time system (together, these are called *Java Virtual Machine*)



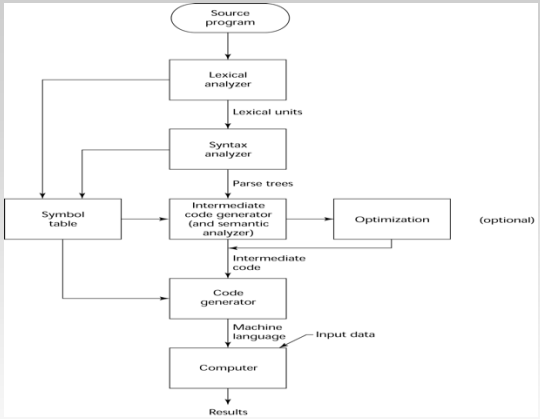
Compiler and Interpreter

- If both interpreter and compiler exists for a language (e.g., Python),
  - programs are first developed using the interpreter,
  - then they are compiled to obtain fast executing programs.

The Compilation Process



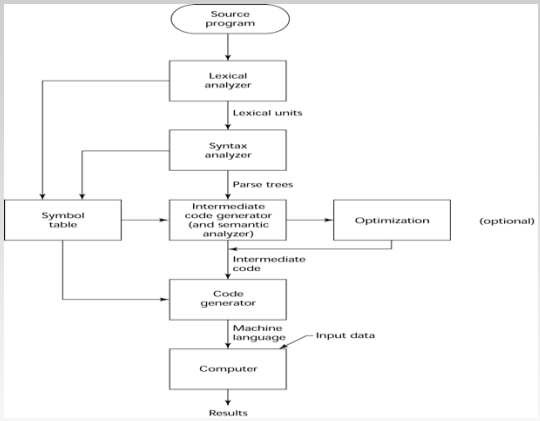
An Overview of Compilation



- Lexical and Syntax Analysis
  - GCD Program (in C)

```
int main() {
    int i = getint(), j = getint();
    while (i != j) {
        if (i > j) i = i - j;
        else j = j - i;
    }
    putint(i);
}
```

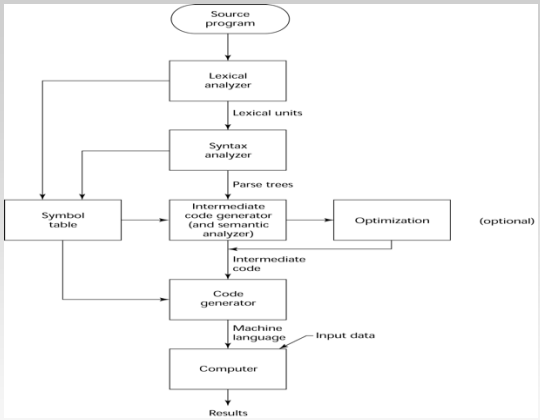
An Overview of Compilation (Scanning)



- Lexical and Syntax Analysis
  - GCD Program Tokens
    - Scanning (*lexical analysis*) and parsing recognize the structure of the program, groups characters into *tokens*, the smallest meaningful units of the program

```
int     main   (   )       {
int     i      =   getint   (   )   ,   j   =   getint   (   )   ;
while   (      i   !=       j   )   {
if      (      i   >        j   )   i   =   i   -       j   ;
else    j      =   j        -   i   ;
}
putint  (      i   )        ;
}
```

An Overview of Compilation (Lexical and Syntax Analysis)



- Context-Free Grammar and Parsing
  - Parsing organizes tokens into a *parse tree* that represents higher-level constructs in terms of their constituents
  - Potentially recursive rules known as *context-free grammar* define the ways in which these constituents combine

An Overview of Compilation (Parsing)

- Context-Free Grammar and Parsing
  - Example (`while` loop in C)

iteration-statement → *while (expression) statement*

statement, in turn, is often a list enclosed in braces:

statement → *compound-statement*

compound-statement → *{ block-item-list_opt }*

where

block-item-list_opt → *block-item-list*

or

block-item-list_opt → ϵ

and

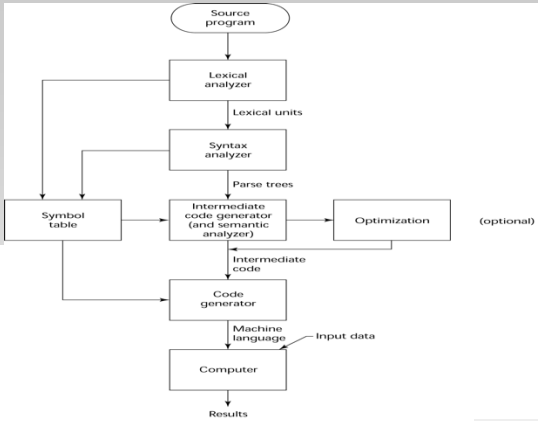
block-item-list → *block-item*

block-item-list → *block-item-list block-item*

block-item → *declaration*

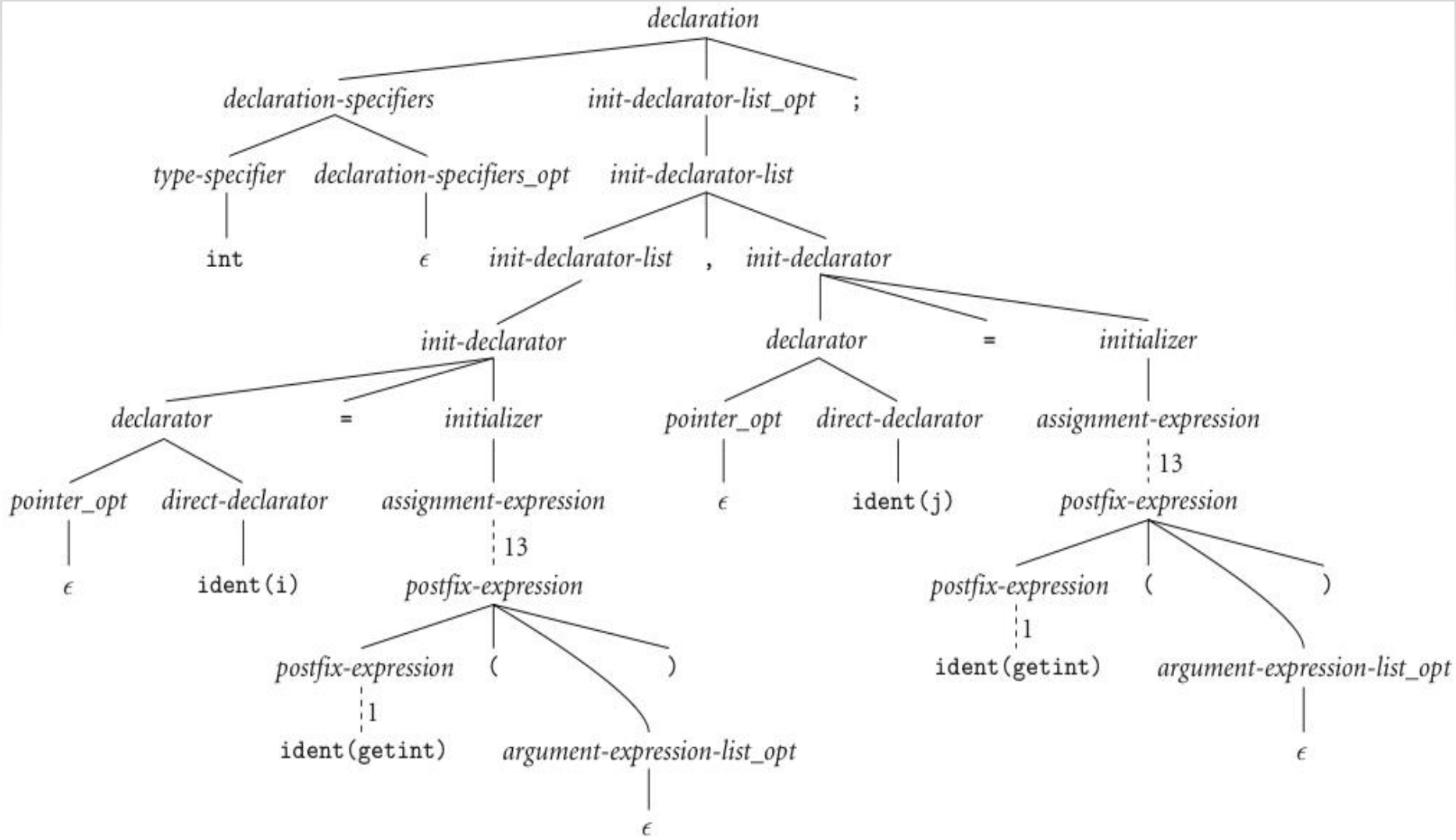
block-item → *statement*

An Overview of Compilation (GCD Program Parse Tree)

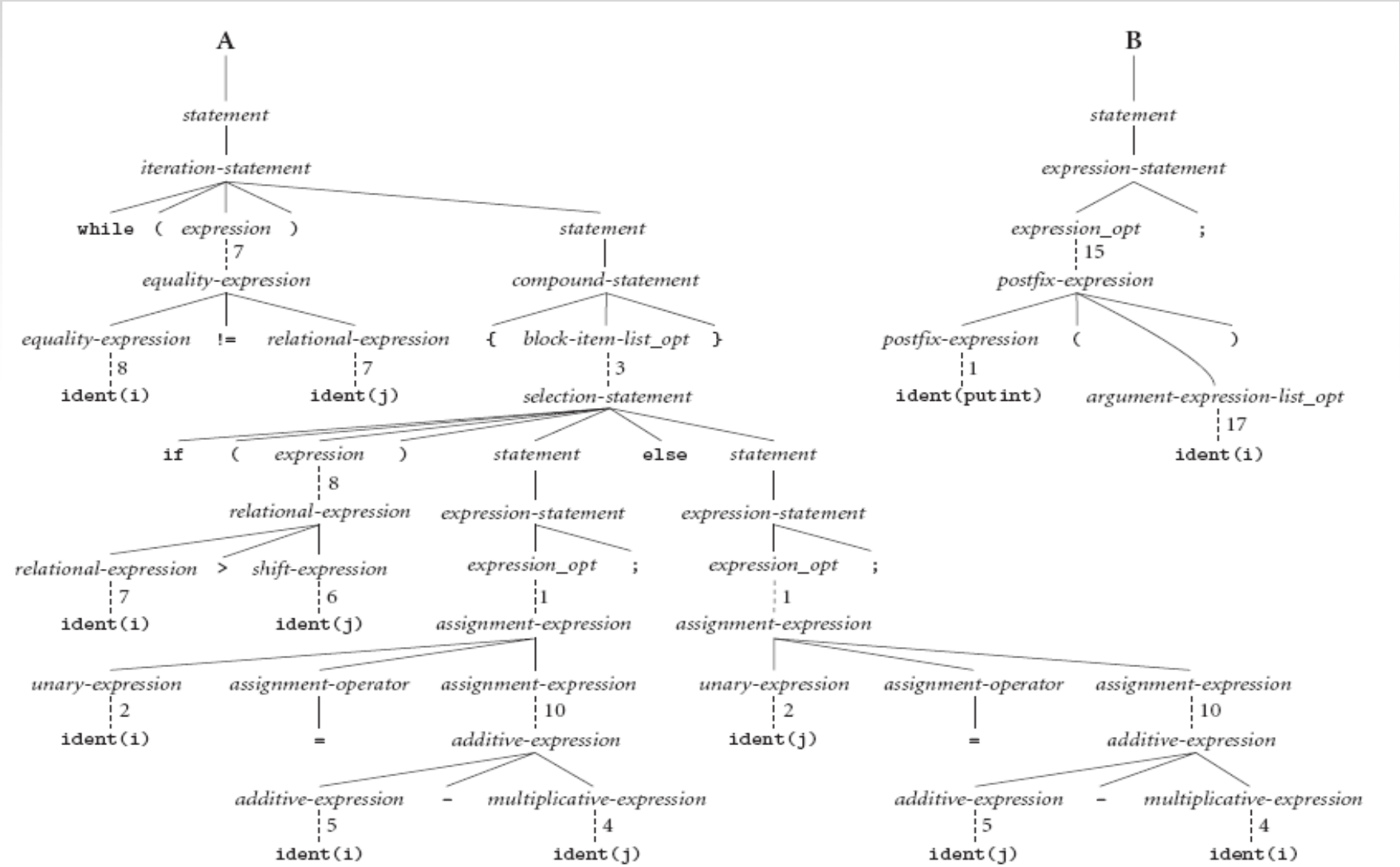


translation-unit
1
function-definition
declarator
declaration-list_opt
compound-statement
pointer_opt
direct-declarator
ϵ
{ block-item-list_opt }
ϵ
direct-declarator (identifier-list_opt)
block-item-list
declaration-specifiers
ident(main)
ϵ
block-item-list
block-item
type-specifier
declaration-specifiers_opt
block-item-list
block-item
B
int
ϵ
1
A
declaration
next slide

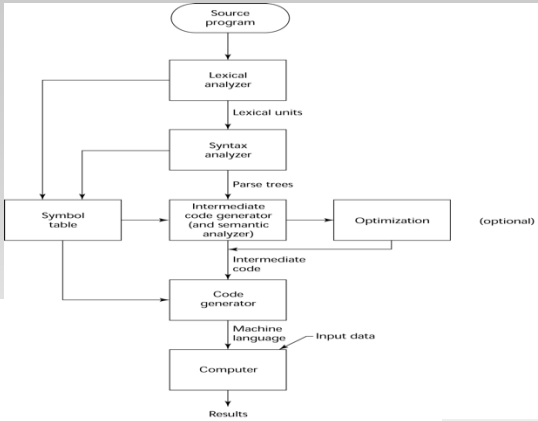
An Overview of Compilation (GCD Program Parse Tree)

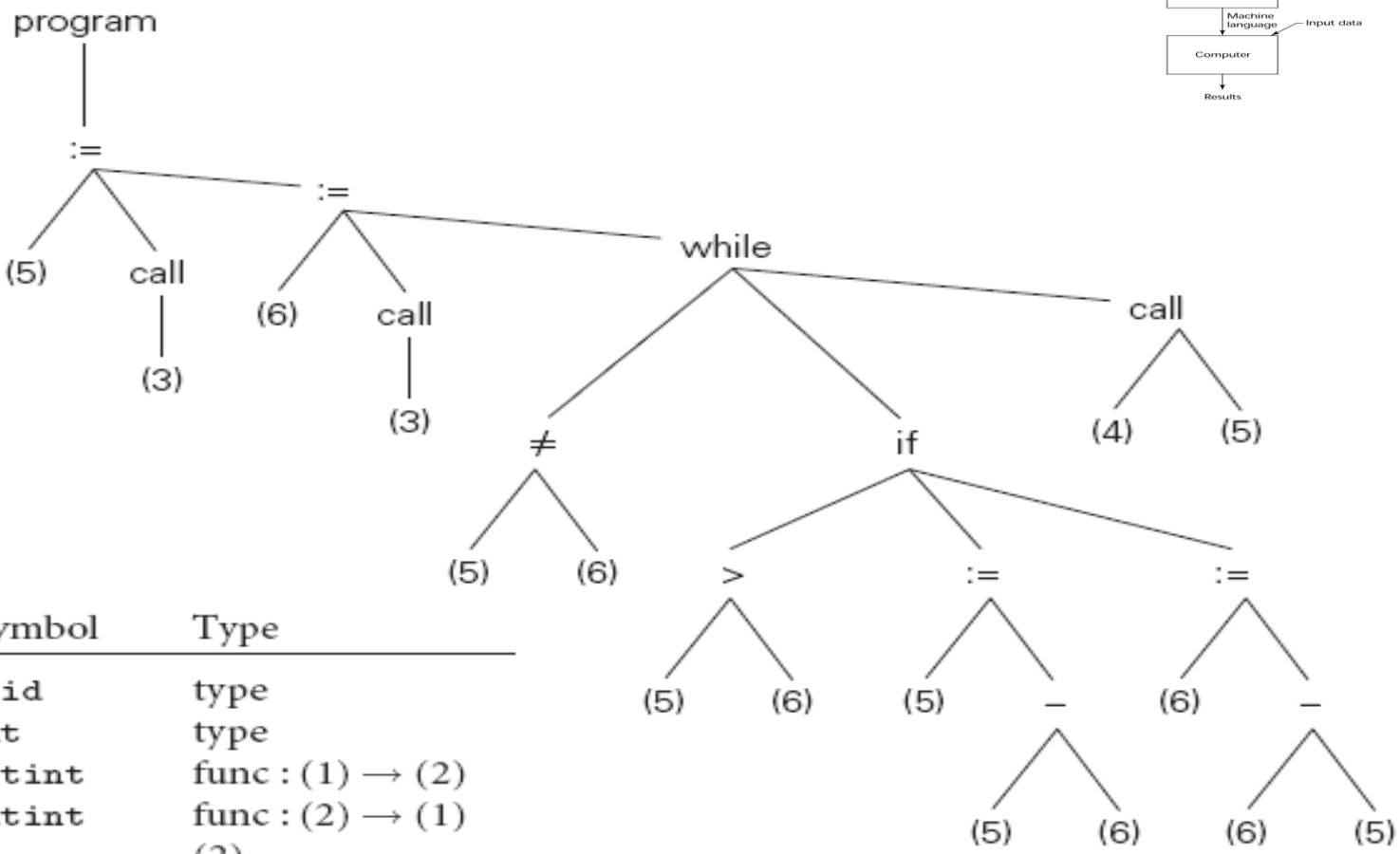


An Overview of Compilation (GCD Program Parse Tree)



An Overview of Compilation (Syntax tree)

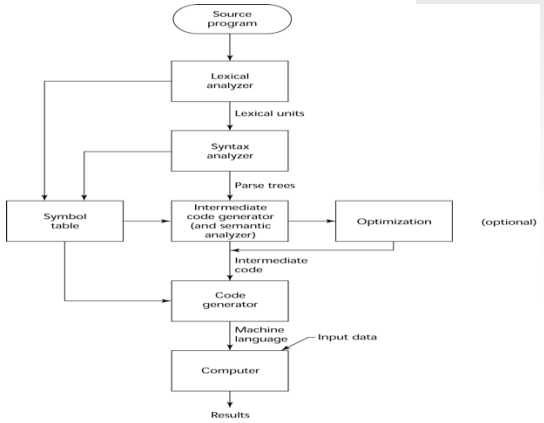




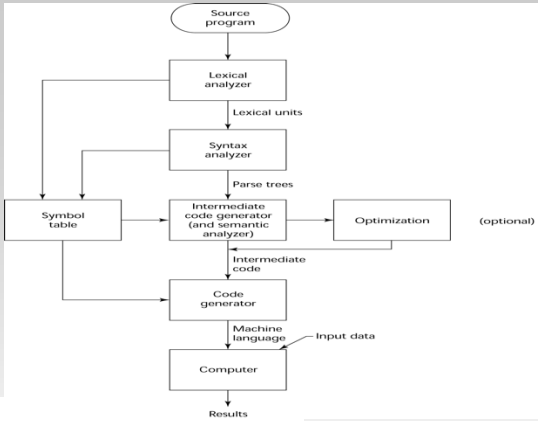
Index	Symbol	Type
1	`void`	type
2	`int`	type
3	`getint`	func : (1) → (2)
4	`putint`	func : (2) → (1)
5	`i`	(2)
6	`j`	(2)

An Overview of Compilation (Target code)

```
     pushl   %ebp                # \
     movl    %esp, %ebp          #  ) reserve space for local variables
     subl    $16, %esp           # /
     call    getint              # read
     movl    %eax, -8(%ebp)      # store i
     call    getint              # read
     movl    %eax, -12(%ebp)     # store j
A:   movl    -8(%ebp), %edi      # load i
     movl    -12(%ebp), %ebx     # load j
     cmpl    %ebx, %edi          # compare
     je      D                   # jump if i == j
     movl    -8(%ebp), %edi      # load i
     movl    -12(%ebp), %ebx     # load j
     cmpl    %ebx, %edi          # compare
     jle     B                   # jump if i < j
     movl    -8(%ebp), %edi      # load i
     movl    -12(%ebp), %ebx     # load j
     subl    %ebx, %edi          # i = i - j
     movl    %edi, -8(%ebp)      # store i
     jmp     C
B:   movl    -12(%ebp), %edi     # load j
     movl    -8(%ebp), %ebx      # load i
     subl    %ebx, %edi          # j = j - i
     movl    %edi, -12(%ebp)     # store j
C:   jmp     A
D:   movl    -8(%ebp), %ebx      # load i
     push    %ebx                # push i (pass to putint)
     call    putint              # write
     addl    $4, %esp            # pop i
     leave                       # deallocate space for local variables
     mov     $0, %eax            # exit status for program
     ret                         # return to operating system
```



An Overview of Compilation (Optimization)




```
        pushl   %ebp
        movl    %esp, %ebp
        pushl   %ebx
        subl    $4, %esp
        andl    $-16, %esp
        call    getint
        movl    %eax, %ebx
        call    getint
        cmpl    %eax, %ebx
        je      C
A:      cmpl    %eax, %ebx
        jle     D
        subl    %eax, %ebx
B:      cmpl    %eax, %ebx
        jne     A
C:      movl    %ebx, (%esp)
        call    putint
        movl    -4(%ebp), %ebx
        leave
        ret
D:      subl    %ebx, %eax
        jmp     B
```

“Hello World” in Different Languages

https://replit.com/languages/python3
https://replit.com/languages/c
https://replit.com/languages/java10

…

Java

```
public class Hello{
  public static void main(String[] args){
     System.out.println("Hello, world!");
  }
}
```

C

```
#include <stdio.h>
int main(){
  printf("Hello, world!\n");
  return 0;
}
```

Python

```
>>> print('Hello World')
```

Prolog

```
?- write('Hello world'),nl.
```

Haskell

```
putStrLn "Hello, World!"
```

C++

```
#include <iostream>
int main(){
  std::cout << "Hello, world!\n";
}
```

Fortran

```
      PROGRAM
      HELLO
      WRITE(*,10)
10    FORMAT('Hello, world!')
      STOP
      END
```

LISP

```
(format t "Hello world!~%")
```

COBOL

```
IDENTIFICATION DIVISION.
PROGRAM-ID. HELLO-WORLD.

ENVIRONMENT DIVISION.

DATA DIVISION.

PROCEDURE DIVISION.
DISPLAY "Hello, world!".
STOP RUN.
```

Perl

```
print "Hello, world!\n";
```

ADA

```
with Ada.Text_Io;

Procedure Hello is

begin

   Ada.Text_Io.Put_Line ("Hello, world!");

end Hello;
```

Assembly Language

```
bdos    equ 0005H   ; BDOS entry point
start:  mvi c, 9    ; BDOS function: output string
lxi     d, msg$     ; address of msg
call    bdos ret    ; return to CCP
msg$:   db 'Hello, world!$'
end     start
```

HTML

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
   "http://www.w3.org/TR/html4/strict.dtd">

<html>

<head>

<title> Hello, world! </title>

<meta http-equiv="Content-Type" content="text/html;
   charset=UTF-8">

</head>

   <body>

    <p>Hello, world!</p>

   </body>

</html>
```